VISPLAY / ENGINEERING HANDOVER

Complete Beginner's Test, Integration & Launch Runbook

Step-by-step instructions for the employer and appointed technical team

VERSION 1.1 • 11 NOVEMBER 2026

Current verdict: NOT PRODUCTION READY. Employer-run tests: ZERO reported.

Use only isolated staging accounts and test payments. Money, privacy and media protection require independent expert sign-off.

Included: Sections A–F source verification, provider setup, user journeys, native-reader testing, remaining G–J work and release decision forms.

Quick navigation

Phase 0 — Stop signs and people to contact

Phase 1 — Safely receive and open the project on Windows

Phase 2 — Create isolated test accounts and record prerequisites

Phase 3 — Supabase: sign-in, email and database staging

Phase 4 — Cloudflare R2: public artwork, private movies and previews

Phase 5 — Paystack and financial core: TEST MODE ONLY

Phase 6 — Deploy two separate staging services (ENGINEER)

Phase 7 — The employer's exact click-by-click web test

Phase 8 — Mobile /reader smoke test (website only; separate from the new native source)

Phase 9 — Security and negative test worksheet

Phase 10 — Finishing Sections F–J before real launch

Phase 11 — Browser, device, international and failure testing

Phase 12 — Full project acceptance register

Phase 13 — Bugs, rollback, security incidents and handover

Phase 14 — Final release sign-off sheet

Tip: Use PDF search for a phrase such as **Paystack**, **Creator Studio**, **DELETE** or **BLOCKED**.

Version: 1.1 — 11 November 2026 **Audience:** VisPlay employer/product owner, a beginner following instructions, and their appointed engineer **Project:** VisPlay web platform, premium streaming, creator publishing, Paystack wallet, and proposed iOS/Android reader app **Critical status:** NOT PRODUCTION READY. The employer reports **zero tests or integrations performed**. Section F now has *source code*, but no signed native builds or device tests; other sections remain outstanding.

> **Read this first:** This is a guided checklist, not permission to deploy real money, erase data, run SQL against a live customer database, or put an app in a store. Work only in **STAGING / TEST MODE**. If a step asks for a secret key, enter it directly into that provider's secure dashboard or the server's private environment settings. **Never** send it in chat, email, screenshots, GitHub or a shared report. A nontechnical employer may perform the labeled **OWNER** steps; an experienced engineer or DBA must perform the labeled **ENGINEER** steps. Stop whenever a required label, permission or screen differs from the instructions.

How to read the runbook

You do not need to know programming to perform the **OWNER** tests. Think of **staging** as a rehearsal stage with test users and pretend money. **Production** is the real public service with real people, protected content and actual transactions. A **test key** connects to a provider's sandbox; a **live key** can affect real money. An **API** is how the screens talk to the server. A **webhook** is a message Paystack sends to the server after a payment. A **migration** changes database structure or records. A **private bucket** is a storage room visitors cannot open directly.

The four status labels

Label	Exact meaning	Who writes it
NOT RUN	Nobody has performed the manual/provider/device step yet	Default for everything below
PASS	Test performed; expected result observed; evidence saved	Person who ran the test
FAIL	Expected result missing, unauthorized access, broken behavior or error	Person who ran the test
BLOCKED	Required environment, permission, section or decision missing; test could not be completed	Person who tried

Evidence rule: After each test, write its ID, date/time, tester, environment URL, device/browser, status, expected/actual behavior, and screenshot/video filename. **Blur email addresses, bank details and personal data**. Never photograph passwords, keys, session cookies or signed upload URLs. A developer's passing unit test does **not** change a manual test to PASS.

Local source verification already performed by developer

- On 11 November 2026, the developer ran source checks covering identity, private media, upload, finance, reader UX and **six mobile navigation-policy tests**, plus the web TypeScript/build steps. These exercise mocks/local routes/source policy, **not provider dashboards or real phones**. Section F's native Expo typecheck and signed builds require separately installed mobile dependencies.
- The developer's original, **not redistributed** source snapshot was assessed with npm run finance:audit; it returned **exit code 2** with blocking sample payment references, mismatched wallet balances, unbound value-bearing legacy identities and an unsupported historic paid payout. **This is the safe, expected STOP result** on that protected input, not a test to bypass. The public handover ZIP excludes that JSON dataset for privacy.

- No employer-run manual tests, real cloud-provider configuration, production database import, actual Paystack payment, mobile-device test or store submission has been reported.

Phase 0 — Stop signs and people to contact

Do not continue toward launch if ANY item below is true

- The **financial audit fails** or there is no signed statement matching every real wallet, purchase, Paystack reference, creator liability and payout; do not mark migration_approved=true to get around it.
- Any full paid film can be downloaded from an unsigned, public R2 URL or watched in full by an unpaid person.
- A seeded admin, creator or account with money gets access because someone signed in using an email that looks the same. Verified identity ownership must be reviewed and bound by the authorized team.
- A creator can publish before the master film is private, preview is generated, scanning/rights review passed, and admin approves.
- A payment is credited twice, debits money without an entitlement, or a refund/payout is marked complete without a provider/bank receipt.
- A user cannot request deletion in the app, deletion unexpectedly forfeits a balance, or deleted users can continue to access their account.
- Apple/Google approval, content distribution rights, moderation or privacy notices are missing.
- Backups and a tested restore, incident contacts and monitoring are missing.

Fill in the contact sheet (do not put secrets here)

Item	Write the non-secret answer
Employer / product decision maker	_____
Lead developer / engineering contact	_____
Database administrator	_____
Security tester / emergency contact	_____
Person allowed to approve real-money launch	_____
Staging website URL (not the live one)	https://_____
Staging server/API URL	https://_____
Staging Supabase project name	_____
Test R2 public/private bucket **names only**	_____
Test Paystack merchant account **name only**	_____
Incident contact phone / approved working hours	_____

Who should do which work?

Label	Safe for a beginner?	Examples
OWNER	Yes, follow screenshots/checklist and stop if unsure	Open sites, register test account, click interface, record behavior, review content rights
ENGINEER	No, supervised operator	Set API URLs, build/release services, store environment variables, test signed uploads, diagnose logs
DBA / FINANCE APPROVER	No; requires independent review	Run SQL, migrate identities, reconcile wallets, activate financial core, restore backups
LEGAL / STORE OWNER	No; qualified reviewer or account owner	Content rights, global market, tax/consumer privacy disclosures, store policy enrolment

Phase 1 — Safely receive and open the project on Windows

OWNER + ENGINEER. Use this phase only on a copy of the cumulative source ZIP. Do not paste source/keys into the original employer production server.

1. In Windows **File Explorer**, create a new folder named C:\VisPlay-Testing (choose an approved non-synced folder if company rules require it). Put the delivered VisPlay_Security_Remediation_Sections_A_to_F_...zip there. **Right-click the ZIP Extract All Extract.** Keep the original ZIP unchanged for rollback.
2. Open Node.js on the engineer's computer. Click the supported **LTS** Windows installer, install the default options, and restart the Terminal. Ask the engineer to verify the source's version/toolchain; do not replace the existing dependency lockfile carelessly.
3. Open Visual Studio Code. Use **File Open Folder** and select the extracted folder that contains package.json, server.ts, src and migrations.
4. In VS Code select **Terminal New Terminal**. You should see a path ending in the project directory. Type node --version, press **Enter**. Type npm --version, press **Enter**. If either says not recognized, stop and ask the engineer to fix the Node setup.
5. The engineer runs npm ci (prefer the included package-lock.json), then npm run test:all. Wait for completion. You want **all six test groups passing**, tsc --noEmit with no error, and the message built. This command does **not** compile Android/iOS; that is a separate mobile step below.
6. The handover ZIP intentionally **does not include** data/visplay_db.json (it may contain personal/financial data). With no file, npm run finance:audit says a snapshot is missing. This is **not a PASS**. Ask the DBA to take an approved, private, backed-up snapshot from the actual source system. The engineer then runs npm run finance:audit -- --input "ABSOLUTE_PATH_TO_APPROVED_SNAPSHOT.json" in a controlled environment. On the developer's original source, the audit returned **exit code 2 with blocking anomalies**. **Write `BLOCKED` for financial launch; do not edit numbers to force a PASS.**
7. Never share the output of .env, a provider API token, or a password. .env.example is only a **template**. Only the engineer may create .env with staging values locally or use hosting secret managers. Do not commit it to Git.

Beginner check P01: Can you identify package.json, .env.example, docs, migrations, and src in VS Code's left Explorer? If yes, write PASS; otherwise ask for help and record BLOCKED.

If the computer has no suitable local tools

Do not guess terminal commands or download random installer sites. Ask the engineer to run Phases 1/3/4; the owner can start at Phase 7 once given a working **staging** link and test accounts.

Phase 2 — Create isolated test accounts and record prerequisites

OWNER: Create/ask for the following **staging-only** identities. Use distinct real inboxes under the company's control, **not someone else's personal emails**. Keep the passwords in an approved password manager.

Test account	Purpose	Current access expected
VIEWER_A	Regular viewer with initially zero wallet	Browsing, preview, own purchases, deletion
VIEWER_B	Separate person to test access isolation	Never inherits A's films or balance
CREATOR_A	Applies to publish films	Cannot upload until reviewed/approved
ADMIN_A	Manages reviews	Only **independently authorized and bound** administrator
DELETION_A	Disposable account used for deletion	No paid orders, no balance, no creator obligations
FUNDS_A	Test wallet balance and blocked deletion	Needs a cleared/safely reconciled test balance

ENGINEER: Confirm a second operator has explicitly assigned ADMIN_A; do not use a seeded user_admin_01 just because the email matches. Make sure test users cannot see each other's data and test accounts cannot affect live customers. Save a private test-identity worksheet without passwords.

Phase 3 — Supabase: sign-in, email and database staging

Status: NOT RUN. Official starting point: Supabase Dashboard and Supabase production checklist.

Create project and collect public values (OWNER with ENGINEER)

1. Open https://supabase.com/dashboard in a normal browser and sign in to **the employer's own team account**.
2. Click **New project** or **New Project**; choose the company organization. Give it an unmistakable name such as VisPlay-STAGING. Select an appropriate region. The database password is a secret: save in a password manager; never add to this runbook.
3. Wait until Supabase says the project is healthy. In the left menu locate **Project Settings** or the newer **Connect / API Keys** area. Find the **Project URL** and **publishable/anon public key**. Only these two may be used as frontend VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY after engineering review.
4. Separately locate the **service-role secret**. **Do not copy it into frontend variables, a screenshot, this document, or a customer-facing tool.** The engineer stores it as the server-only SUPABASE_SERVICE_ROLE_KEY in the staging API host's secure environment settings.
5. Make sure the server's SUPABASE_URL and frontend's VITE_SUPABASE_URL refer to the **same** staging project. The service-role key must match that project.

Enable confirmed email and redirects (OWNER can read, ENGINEER configures)

1. In the Supabase left navigation, click **Authentication Providers / Sign In**; open **Email**. Turn on **Confirm email**. This application rejects a new account if the setting is disabled.
2. Open **Authentication URL Configuration**. Set **Site URL** to the staging website's secure https://... origin. Add allowed redirect URLs for the site's / and /reader entry points as appropriate (copy the exact route/redirect patterns from Supabase redirect documentation). Do not use blanket wildcard domains in production.
3. In **Authentication SMTP Settings**, configure a company-controlled sender and transactional mail service. Read Supabase SMTP. The default testing mail service may be limited; **do not use it as a production sender**.

4. **ENGINEER:** Optionally set up Google login on the **web** using Google provider instructions. The /reader view intentionally hides embedded Google OAuth until a proper native system-browser flow is built in Section F.
5. **OWNER:** With the staging site later running, use **Join Free Create Account** with VIEWER_A. Submit. Check the inbox/spam for verification email. Click the link **once**. Sign in and confirm it shows the right test user. Check an unverified email cannot obtain a logged-in app session.

Apply migrations: ENGINEER/DBA only, never a solo novice action

1. Confirm the project header reads **VisPlay-STAGING**, not a live or unrelated project. Make an export/snapshot first and record its restore location.
2. Read, peer-review and test these source files in this order: migrations/20261107_visplay_upload_sessions.sql, then migrations/20261107_visplay_financial_core.sql.
3. In Supabase **SQL Editor New query**, the **DBA** inserts/reviews the SQL from each file and executes it on a disposable staging database after backup and change approval. The owner must **not** paste arbitrary SQL from messages into a live project.
4. **DBA:** Verify upload-session table permissions and RLS. Anonymous and normal signed-in clients must not directly read/update service-role-only upload sessions or call protected financial routines.
5. **DBA:** Keep visplay_finance_control.migration_approved set to **false**. This deliberately blocks money features until a separately approved, reconciled test dataset is imported. SQL schema installed financial system activated.
6. **FINANCE + DBA:** Reconcile real legacy finances *outside* the test-account activity: Paystack settlement exports, bank payout receipts, old wallets, legacy identities and purchases, payout/reserve events. Sign the reconciliation. Only an authorized independent reviewer can approve the control flag. The supplied source currently **fails** this gate.

Expected: Verified email works with test accounts, OAuth follows allowed rules, ordinary users cannot become seeded admins or inherit old wallets, SQL permissions deny direct client calls. **Stop** at unexpected auto-confirmation, missing mail, identity takeover, or unexpected positive balance.

Phase 4 — Cloudflare R2: public artwork, private movies and previews

Status: NOT RUN. Official Cloudflare R2 docs and CORS docs.

1. **OWNER + ENGINEER:** Sign in to Cloudflare Dashboard. In the left menu open **R2 Object Storage Overview** and choose **Create bucket**. Make a public-artwork bucket, e.g. visplay-staging-assets. Make a **different** private-film bucket, e.g. visplay-staging-private. The names must not match each other.
2. **ENGINEER:** On the private bucket open **Settings Public access / Public development URL** (label may vary). Ensure its r2.dev access is **disabled** and it has **no custom public domain**. Do not assume “private” based on its name. Try accessing a private object while signed out; it must return unauthorized/not found, not the film.
3. **ENGINEER:** In R2 token/permission management, create bucket-scoped access suitable for server and isolated worker; choose the least privileges possible. Place R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET_NAME, R2_PRIVATE_VIDEO_BUCKET, and R2_PUBLIC_BASE_URL into the **server/worker staging secret settings**, never a React VITE_ variable. Separate staging and production tokens.
4. **ENGINEER:** In the private bucket's **CORS** settings, allow only the exact staging web origin(s), required PUT/HEAD, signed-upload Content-Type / Content-Length headers, and expose ETag for multipart. Test actual browser preflight/part uploads. Do not set all origins to * as a shortcut.
5. **ENGINEER:** Set **lifecycle rules only for `staging/`**: remove abandoned staging objects after approximately two days; abort uncompleted multipart uploads after approximately one day. Never apply these cleanup rules to videos/ or previews/.

6. **ENGINEER:** Provision a separately isolated worker with FFmpeg, FFprobe, the configured updated malware scanner (e.g. clamscan) and limits on memory, CPU, duration and temporary disk. A correctly configured worker can run npm run media:preview -- <MOVIE_ID> for one safely uploaded test film. The worker returns a preview_private_key, which is then **attached by the authorized administrator** in the Super Admin Secure Media control.
7. **OWNER:** Use a film that the employer **owns or explicitly licensed for testing**. Ask CREATOR_A to submit it. Confirm the preview does not exceed about five minutes; its private master is never accessible from a direct public link.
8. **ADMIN_A:** Open **Super Admin Portal movie review queue**. Verify the **Approve & Publish** button remains disabled until the private master, generated preview, rights and review checks are complete. Attach the preview under Secure Media with engineer assistance, review content and then publish.

Important limitations: The preview tool currently handles films one by one, not a production queue. The employer must fund/approve a worker job queue, monitoring, bounded storage cleanup and controlled parallel transcoding before a global launch. The media gateway also needs real-world concurrency and CDN cost tests. **No DRM is included.**

Test media ownership, never bypass paid films

- **VIEWER_A unsigned:** Can play only the independently generated short preview on a published film.
- **VIEWER_B unpaid:** Can seek within their preview, but cannot request the full film, even by changing URL parameters or browser state.
- **VIEWER_A legitimately owns test film:** Can play full length and seek after an authorized staging purchase/entitlement.
- **Unpublished film:** Stranger receives no preview or full access.
- **Legacy public film URL:** Must stop working after the supervised private migration and CDN invalidation. Do **not** delete original masters until verified backup/private copies and entitlement reconciliation are signed.

Phase 5 — Paystack and financial core: TEST MODE ONLY

Status: FINANCIAL ACTIVATION BLOCKED. Use Paystack Dashboard and official test payments, API keys/webhooks, and verify payment.

Nontechnical owner dashboard steps (no money moved)

1. Sign in to the company's own Paystack account. Verify you are in **Test Mode**. If the dashboard says **Live**, stop.
2. Open **Settings** and find **API Keys & Webhooks** (labels may change). Ask the engineer to store the sk_test_... value only in the server's staging PAYSTACK_SECRET_KEY secret setting. **Never send it to the frontend or this runbook.**
3. Ask the engineer for the **exact public staging API URL**. On the Paystack Test configuration, set the webhook URL to https://YOUR-STAGING-API/api/paystack/webhook. It must reach the API service directly or through a verified same-origin rewrite with raw-body signature preservation.
4. Ask the engineer to set PAYSTACK_CALLBACK_URL to the approved HTTPS **web checkout** return URL. It must use the same expected staging origin; the reader app has no Paystack checkout.
5. Have VIEWER_A run the ordinary **web** checkout only after the DBA has produced a clean, approved **isolated staging ledger**, the financial migration gate is approved by authorized signers for that isolated dataset, and a test Paystack account is connected. Use only Paystack's **published sandbox test cards**, not a real bank card.

What a safe test payment should do

1. **OWNER:** Open https://STAGING-WEB/ in Chrome. Click a published test film. Watch the preview. At the paywall click **Unlock Full Movie for ₦100**. Sign in if prompted. The browser should open official Paystack in test mode.

2. Use the provider's stated **successful test payment** instructions. Do not enter an actual bank card. After redirect, the movie should show as purchased **only once**, and the account should be able to play the full film. Save only the nonsecret reference/result screenshot.
3. Try a **failed/cancelled test payment**. Expected: no unlock and no wallet credit. Browser error must not reveal keys or raw user data.
4. **ENGINEER:** Compare the same reference and amount in the staging Postgres tables and Paystack test dashboard. Verify the x-paystack-signature webhook is checked over raw bytes. Send a valid webhook/replay through an authorized isolated test harness; one payment must never create two credits, two entitlements or two creator allocations.
5. **OWNER:** Open **My Library & Purchases**; verify the film is recorded for A. Sign in as B; verify **no** entitlement. For wallet test, use **Fund Wallet / Deposit Coins** in the **web** interface with approved test funds; do not assume a UI success means provider settlement occurred. Compare signed ledger evidence.
6. **ENGINEER + FINANCE:** Test two simultaneous purchases, refund case, webhook-first settlement, callback never arriving, simultaneous balance spends, and conflicting payout reservations **in a separately resettable staging dataset**. These are not safe beginner click tests on live finance.

Important missing financial features — do not claim complete

- The patch provides idempotent purchase/deposit handling, creator earnings records and payout **reservation**, but it **does not actually send creator payouts to banks**.
- Duplicate already-owned purchase charges may open a refund case; there is **no full tested Paystack refund/chargeback fulfillment flow** yet. No real-money launch until built, independently checked and reconciled.
- Daily accounting comparison of Paystack settlement, bank movements, wallets, creator receivables, refund/chargeback cases is pending operational integration.
- Legacy sample references and balances are **not trustworthy production value**. Do not transfer or redeem them without evidence and a signed migration.
- International currency, foreign tax and territorial payment-provider support are not implemented by an NGN-only wallet.

Phase 6 — Deploy two separate staging services (ENGINEER)

Status: NOT RUN. Official Render web service and Vercel env variables.

1. Use a **private company Git repository** that excludes .env, .git from nested archives, node_modules, dist, real data/ snapshots, customer movie masters and credentials. The handover deliberately omits the original data/visplay_db.json; **do not reintroduce it into a public repository** or treat demonstration records as a production database.
2. In Render Dashboard, select **New Web Service**. Connect the approved repository/branch and name it clearly visplay-api-staging. Choose a reliable plan with persistent operational monitoring; the supplied render.yaml contains plan: free, **not** a production service recommendation. The engineer validates Node/build/start commands on the selected host.
3. In the Render service open **Environment**. Enter server-only staging secrets from .env.example: SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, JWT_SECRET, TRUSTED_WEB_ORIGINS, PAYSTACK_SECRET_KEY, PAYSTACK_CALLBACK_URL, and the R2 variables. Follow provider docs for encrypted environment values. Ensure NODE_ENV=production on deployed service after the safe staging financial gate is genuinely approved, with no demonstration data.
4. Deploy and open **Logs**. A startup refusal for unapproved financial migration is a **correct fail-closed security control**, not a reason to disable it. Ask the finance/DBA team to fix the cause. Never use migration_approved=true on an unreconciled dataset to make Render show healthy.

5. In Vercel Dashboard, select **Add New Project**, import the approved repo. Set build framework/deploy branch for the frontend. Under **Settings Environment Variables**, add the two **public** VITE_SUPABASE_... variables pointing to the same staging Supabase project; do not add server secrets as VITE_ keys.
6. **Critical configuration:** The repository's vercel.json currently rewrites /api/:path* to https://visplayapp-2-0.onrender.com/api/:path*, and /uploads/:path* to the old backend. The engineer must change it to the **actual approved staging API host** and remove/disable the obsolete /uploads public proxy after verifying migration. Otherwise the staging frontend can accidentally contact the wrong server. Review TRUSTED_WEB_ORIGINS to allow only the correct HTTPS staging frontend.
7. Deploy the frontend. Open the assigned Vercel **Preview / Production-like staging URL**. Test https://STAGING-WEB/ and https://STAGING-WEB/reader separately. Verify both load, route refresh works, API network calls go to the intended staging backend, and emails return to the correct path.
8. **ENGINEER:** Set up error logging with personal-data redaction, uptime/5xx alerts, webhook 503 alerts, audit logs, daily database/object snapshots, and a **tested** restoration to another staging instance. Agree an incident response contact and rollback steps before inviting testers.

If deployment stops because SQL/finance is unapproved, record BLOCKED and finish the read-only tests first. Do not point the frontend at an unrelated live server to make screenshots look complete.

Phase 7 — The employer's exact click-by-click web test

OWNER. Do this only after the engineer provides a secure, working staging URL and safe test accounts. Use a fresh Chrome or Edge browser, then repeat important journeys in Safari/Firefox. Each line below starts as NOT RUN.

7A — Home, search, films and creator discovery

1. Open a new tab, type the entire https://STAGING-WEB/ address in the address bar, press **Enter**.
2. Confirm the VisPlay logo, featured films, creator discovery and movie cards render. Scroll to the bottom and check the footer's privacy/account-deletion link. Broken images, blank panels, errors, or unrelated film titles are FAIL.
3. Click **Join Free** in the top navigation; choose **Create Account** if the modal offers tabs. Register VIEWER_A, verify its mailbox, sign in. If the email link fails, mark FAIL and save the error message (not the email verification token).
4. Click a movie poster/title. Confirm the film title and description are visible. Click **Play** if offered and watch the permitted preview. Move the progress slider and verify the player stays within the preview range for unpaid viewers.
5. Browse **Creators** and the **Bundles** tab. Click a creator profile and a bundle; check the displayed content belongs to that creator/bundle. An unpurchased bundle should not reveal full episodes.
6. Click **My Library & Purchases** from the user menu. Confirm only A's unlocked films and owned bundles appear. Verify the wallet balance starts at zero for a truly new user.

7B — Profile, account settings and deletion

1. Open the user/profile menu in the top-right, click **Account Settings**. The page should open the **Profile** tab, not send you back to the Library tab.
2. Change a non-sensitive test display name, click **Save Profile Changes**, refresh and check it persists. Never enter someone else's identity details.
3. Scroll to **Delete Account**. The text must say deletion is blocked when there is a positive wallet or unsettled creator obligations; it must **not** say users permanently lose their money.
4. Use test account DELETION_A with zero balance. First type an incomplete confirmation; **Permanently Delete My Account** stays disabled. Then type DELETE (capital letters) and click the button. In the browser confirmation dialog choose **Cancel**; account must remain.

5. Repeat with a disposable eligible account, choose **OK** only after the tester has permission to delete this disposable test identity. Expected: session cleared, account removed as allowed by retention policy, no access to former profile or private films. Do not use a real person's account.
6. On test account FUNDS_A with an independently verified positive **staging** wallet balance, attempt deletion. Expected: an error explains how to settle/withdraw the balance; the account and money are preserved. **No deletion or forfeiture**.
7. In a logged-out browser, open https://STAGING-WEB/account-deletion.html. Confirm a clear public deletion explanation is readable and links to the app. A public external deletion initiation/request flow and support path may still need completing in Section G for Google Play compliance.

7C — Creator application, upload, moderation and access

1. Sign in as CREATOR_A. On the **web** top menu choose **Become a Creator / Apply** (exact wording may vary) and submit a test application with licensed test content. Expected: **under review** status; no immediate publishing rights.
2. Have ADMIN_A, previously bound and independently authorized, sign in via a separate private browser. Click **Super Admin / Super Admin Portal**. Go to creator applications and click **Approve Creator** on A's deliberate test application. A must now gain access to **Creator Studio**; unrelated users must not.
3. Sign in as A. Open **Creator Studio Upload Film / Upload New Movie**. Enter title, synopsis, genre and licensed distribution-rights declaration. Under **Upload Movie Video File (MP4)** choose a known safe, clean H.264/AAC test MP4 (up to the app's stated 500 MB). Add a JPEG/PNG poster below 10 MB. Click **Submit Movie to Moderation**.
4. Wait for upload progress and expected **submitted to Super Admin moderation queue** message. **Do not claim the film is published yet**. In a test browser intentionally select an invalid file extension or oversized file and confirm a clear rejection. Engineers must separately test multipart/retry/corrupt content on R2.
5. **ENGINEER** runs the isolated malware scan and preview process. **ADMIN_A** attaches the verified private master/preview in the Secure Media review area and, after confirming rights/content and correct source account, uses **Approve & Publish**. The film should appear in public catalogue only then.
6. As an unsigned browser user, attempt to view the newly published full film without buying: only preview should play. As a legitimately entitled A/B user, full film should play. Record two separate results.

7D — Web purchase, wallet, failure and duplicate charge

BLOCKED unless Phase 5 isolated financial staging and Paystack test mode are approved.

1. Sign in as VIEWER_A on https://STAGING-WEB/ (not /reader). Open a published test movie you do not own; watch preview and click **Unlock Full Movie for ₦100**.
2. Confirm the checkout is hosted by Paystack test mode, with the correct product/amount. Use only Paystack's documented test card. Wait for provider return. The film should appear once in A's library and play in full.
3. Refresh the page, sign out and back in. The same purchase must persist. Sign in as B and confirm B cannot watch the full film.
4. Attempt a cancelled/failed test transaction on a **different** item. Expected: no entitlement. Do not treat browser success messages alone as evidence of settlement.
5. On the web open **Fund Wallet / Deposit Coins** and use an approved Paystack test deposit. Confirm exactly one recorded credit and same result after refresh. **ENGINEER** must verify database rows and webhook idempotency.
6. **ENGINEER/FINANCE only:** In a controlled dataset test duplicate webhooks, two checkouts, chargeback/refund case, insufficient balance and competing payout reservations. Compare provider references with canonical SQL. Owner marks these BLOCKED until engineer evidence is attached.

Phase 8 — Mobile /reader smoke test (website only; separate from the new native source)

OWNER. This phase tests the **mobile-reader route in a browser**; it is **not** an App Store/Play Store acceptance test.

1. On a phone's **Safari** (iPhone) or **Chrome** (Android), type https://STAGING-WEB/reader. Press **Go**. You should see a mobile-friendly VisPlay reader. Do not use the root / route for this check.
2. Scroll the home screen, search/browse movies and creator profiles, open a film, and play the **short free preview**. The screen must never show **Fund Wallet**, **Deposit Coins**, **Unlock Full Movie for ₦100**, **Buy Bundle**, Paystack, or a "Go to website to purchase" button. Taking an unpaid user to a browser checkout is **not** universally allowed by store policies.
3. Open **Bundles** and an unowned bundle; the reader must show a neutral description, never an actionable digital-product checkout. Open the bottom **My Library**; it must work and show only previously authorized items.
4. Sign in via email using a verified VIEWER_A (Google sign-in is intentionally hidden in reader until native auth is built). Open a film that A previously purchased **on the independent website**. Full playback should be available. Sign in as B, who did not purchase it; only preview should play.
5. Open the profile menu **Account Settings**. Verify **Delete Account** instructions and typed DELETE confirmation are accessible in portrait and landscape. Cancel deletion during this usability test.
6. Tap every major button with a finger, check no text is cut off on a narrow device, pinch-zoom works, and the bottom navigation does not overlap iPhone gesture indicator/safe-area. Use mobile screen reader/large font settings; ask QA to document issues and retest.
7. **ENGINEER:** Test redirect, refresh, back gesture, logout, deep link, and error state in the **new Section F native source separately** to ensure the shell cannot escape /reader into commercial / views. This browser test alone cannot prove native navigation interception.

Critical store compliance note

Apple and Google generally require their permitted billing systems for digital films watched in an app, with territory-specific alternatives only when their separate exceptions/programs apply. **A button opening Safari/Chrome to a Paystack checkout is not a blanket exemption.** The default proposed reader app must be **payment-free**, with no external purchase promotion, until store-specific lawful eligibility has been confirmed and approved. Apple's guideline 4.2 also requires more than a minimally useful website wrapper in many cases. No one can guarantee immediate approval.

Phase 9 — Security and negative test worksheet

OWNER can carry out simple separate-account and cancel-flow checks. **ENGINEER/independent tester** controls all API probing, race tests, fuzzing, load testing, file-malware fixtures or pen tests, within an approved written staging scope. Never attack a live service as part of a beginner runbook.

ID	Actor	Simple action	Required observation
S01	OWNER	New verified account signs in	No legacy wallet, creator or admin role inherited
S02	OWNER	Unverified registration opens private library	Rejected; verification required
S03	OWNER	VIEWER_B looks for A's paid film	B sees no full content/entitlement
S04	OWNER	Sign out then use browser Back on profile	No private account data accessible
S05	OWNER	Cancel account deletion dialog	Account remains active
S06	OWNER	Delete account with positive test balance	Blocked with balance preserved

ID	Actor	Simple action	Required observation
S07	ENGINEER	Forge/omit API session for purchase/upload/admin	HTTP unauthorized/forbidden; no state change
S08	ENGINEER	Change object IDs across two users	No IDOR/BOLA; correct isolation
S09	ENGINEER	Direct GET private R2 master from outside	403/404; no data
S10	ENGINEER	Modify stream URL mode=full when unpaid	403; no full media
S11	ENGINEER	Replay the same Paystack webhook/reference	Exactly one settlement/entitlement
S12	ENGINEER	Fake signature/currency/amount	No credit/entitlement; alert
S13	ENGINEER	Rename ZIP/SVG as MP4/JPG	Rejected or sanitized; never publicly accessible
S14	ENGINEER	Interrupted multipart upload, retry/abort	No unintended completed asset; cleanup safe
S15	ENGINEER	Disable malware scanner in production-like worker	Preview generation refuses to proceed
S16	ENGINEER	Remove finance migration approval in isolated environment	Financial operations fail closed
S17	ENGINEER	Examine API JSON and browser network responses	No service-role keys, R2 secrets, master URL or passwords
S18	ENGINEER	Two parallel purchases with one wallet balance	Exactly one valid debit; nonnegative wallet

Phase 10 — Finishing Sections F–J before real launch

These are not just tests. They are **development and operational deliverables not fully completed by source patches alone**.

F — Actual React Native iOS + Android reader applications (SOURCE PREPARED; BUILDS/DEVICES NOT RUN)

What now exists: Open the mobile/ folder inside the delivered source. It contains an Expo/React Native app: native Explore / Saved / Settings tabs, a real native **preview-only** video player, local bookmarks, the OS Share button, safe reader navigation and account-deletion access. Unlike the independent payment website, this reader must have **no film checkout, Paystack button or "pay on our website" promotion**. The mobile source is **not yet compiled, installed, signed or independently tested**.

For the OWNER (you can follow and watch; let the engineer type commands):

1. Before doing anything on a phone, ask the engineer to write the actual **staging** website address (not production) on the contact sheet. Open it on your PC with /reader at the end. If it is unavailable, stop; the mobile app needs a live staging reader.
2. In File Explorer open the extracted package. Open the mobile folder. You should see App.tsx, app.json, README.md, assets and .env.example. This tells you the native project exists, **not** that it has passed a phone test.
3. Ask the engineer to install **Node.js 24 LTS**, approved Visual Studio Code, and (if signing) Expo/EAS CLI. In the VS Code **Terminal New Terminal** run cd mobile Enter, npm install Enter, npx expo install --fix Enter, npm run typecheck Enter, and npx expo-doctor Enter. All must finish without an unresolved error. If you are not an engineer, do not change version numbers to make an error disappear.
4. Ask the engineer to copy mobile/.env.example to a **new** mobile/.env locally. Replace the sample address with EXPO_PUBLIC_VISPLAY_READER_URL=https://YOUR-APPROVED-STAGING-DOMAIN/reader. This is a public URL only,

not a secret. If it is missing or invalid, the app deliberately refuses to connect. Do not copy it to a cloud public page. .env must remain outside the ZIP/GitHub.

5. To try it live while developing, engineer types npx expo start. On a compatible test phone, install/open the Expo Go app using the official app store, scan the QR code generated by Expo if compatible, or use an engineer-made custom development build. If Expo Go or the QR code fails, record BLOCKED; ask the engineer to verify SDK support, Wi-Fi and network mode.
6. For an **Android installable internal APK**, engineer signs into Expo with npx eas-cli@latest login, configures build with npx eas-cli@latest build:configure, then runs npx eas-cli@latest build --platform android --profile preview. The engineer must first confirm the provisional Android package ID, signing account and staging URL. In the completed Expo build dashboard, the owner follows the official **Install** link on a company test Android device. An APK is **not** evidence of Google Play approval.
7. For **iPhone**, the owner/employer first enrolls in the official Apple Developer Program; the engineer must register an **owned** bundle ID, configure Apple certificates and produce a correctly signed iOS build. To test through TestFlight, the engineer uploads that build to App Store Connect and invites approved testers via **TestFlight Internal Testing**. If the company has no developer membership, mark BLOCKED until it does. Do not claim TestFlight exists just because the source contains iOS configuration.
8. Open the native app. On the bottom bar tap **Explore**, then a published test film. Tap **Play preview in native player** and verify it uses device playback controls and ends within five minutes. If the preview fails, ask whether private R2/preview-worker setup in Phase 4 was actually done. Mark BLOCKED until the provider works; never replace the clip with a public full movie.
9. Return to the film. Tap **Save to this device**, then the native **Saved** tab. Check the film title appears. Tap **Open film**, **Share**, and **Remove**. Share should contain only a safe /reader?watch=... link. Turn Wi-Fi off: Saved still shows its bookmarks; no full film is stored or unlocked offline.
10. Tap the native **Settings** tab **Open my library**. Return to Settings **Account settings / Delete account**. Sign in with the verified VIEWER_A email if necessary. Open the account deletion control: check Cancel keeps the account. Test final deletion only on the disposable DELETION_A account with a confirmed zero balance. An account with money must be blocked without losing funds.
11. Ask a tester to check the phone's **Back** button, OS app background/resume, font size, screen reader, landscape, loss/recovery of network, a fresh install, and different Android/iPhone sizes. Have the engineer check that external links, checkout, popups and forged deep links are blocked. Take safe evidence screenshots.
12. **Record F01/F02/F03 as `NOT RUN` until installed-device proof exists.** Section F's native Google login remains **NOT BUILT**: the current reader supports verified email sign-in only. A Google-only user must not be instructed to sign in through an embedded Google WebView. Qualified engineers must add system-browser OAuth with state/PKCE and a secure server session handoff; retest account takeover before launch.

What you should see in the app: "VISPLAY READER" at top; **Explore / Saved / Settings** at bottom; film-preview and bookmark buttons on a film detail; Settings contains **Account settings / Delete account**. There must be no digital-film purchase prompt or checkout link. The web commerce site / remains separate and is **not an approved in-app purchase alternative**.

STORE OWNER: Later, use official **Google Play Console Testing Internal testing** and **App Store Connect TestFlight Internal Testing**, after engineer-signed store-compatible builds exist. Review device logs and complete store privacy, content rating, deletion and payment-policy declarations. Physical testing, native Google OAuth, accurate legal policies and Apple/Google decisions are release gates; **do not submit to public production yet**. The longer F test matrix is in docs/SECTION_F_NATIVE_READER.md.

G — Trust, rights, privacy, support and user safety (NOT STARTED)

Build visible **Report** on films/comments/channels, **Block creator/user**, moderation case queue, response SLAs, spam/abuse/rate controls, rights attestation + takedown, repeat-infringer procedure, child/age-sensitive content controls, creator appeals, support contacts, and lawful privacy/data retention. Publish accurate **Terms**, **Privacy Notice**, **Content/Creator Policy**, **Refund Policy**, and **Support & Account Deletion** information. Confirm Google off-app public deletion mechanism and Apple in-app deletion with the actual product. Track moderating staff access and actions. Users must be able to contact a real support channel, not a placeholder.

OWNER click path after G: Open a movie click **Report** choose reason Submit receive confirmation; open a user/channel **Block** verify the content/contact behavior; admin opens moderation queue views report resolves or escalates with log. Check public privacy/support/deletion links in an unsigned browser.

H — Production integrations and operations (NOT STARTED)

Bring up separate staging and production accounts/projects; finish Supabase migrations and approved identity/finance imports; provision R2 plus isolated worker queue/virus updates; configure Paystack test/live isolation, refunds, disputes, provider settlement reconciliation and bank payout completion; build CI/CD and signed image/artifact promotion; monitor and alert; run backup **restore**; configure WAF/rate limiting/incident playbooks. Migrate legacy public media without exposing ownership keys. Do not rely on the Render free plan for real customer traffic.

OWNER sign-off: Engineer demonstrates live dashboards with staging labels, one successful restore into a disposable environment, an alert drill and one complete refund/creator payout simulation with ledger/provider evidence; independent finance verifies balances. Only later permit a separately approved controlled live pilot.

I — Truly global market (NOT STARTED)

Write a **country-by-country launch matrix**: where each film can legally stream, subtitle/localization requirements, price and minor unit/currency, PSP availability, tax/VAT/sales tax, refunds/consumer rights, sanctions, app distribution, data protection, accessibility, rating and copyright. Build locale formatting, multi-currency ledgers if approved (not just exchange-rate display), geo-rights controls, captions and accessible video player. Get qualified legal/finance sign-off per market.

J — Independent QA, store review and staged release (NOT STARTED)

An independent tester runs the entire case matrix with annotated evidence; security reviewer performs authorized penetration test; accessibility auditor checks keyboard and screen readers; load/performance engineer tests peak streaming/Paystack/webhook throughput; privacy/legal and finance sign release. Upload signed native builds for real TestFlight and Google Play internal testing, complete App Privacy/Data Safety/account deletion/disclosures and respond honestly to reviewers. Use a small monitored pilot, rollback capability and gradual rollout. **Approval is ultimately decided by Apple/Google reviewers and is not guaranteed.**

Phase 11 — Browser, device, international and failure testing

OWNER + QA. Minimum recommended matrix; do not invent evidence for devices unavailable to the team.

ID	Device/network	What to do	Pass condition
D01	Windows Chrome + Edge	Home, browse, auth, preview, profile	No blank page or console-visible error to user
D02	macOS Safari / iPhone Safari	/reader, auth, media seek, keyboard/VoiceOver	All reachable, no trapped focus, touch usable
D03	Android Chrome on entry-level device	/reader, 4G, low memory, landscape	No crashes, tiny controls or overlay obstruction

ID	Device/network	What to do	Pass condition
D04	Slow/unstable mobile network	Start preview, pause, disconnect, reconnect	Clear recovery; no unintended paid unlock
D05	Browser incognito unsigned	Public catalogue, private film URL	Public metadata only; full master not exposed
D06	200% zoom / large text	Film cards, nav, account deletion	Text not clipped and buttons operable
D07	Keyboard-only + screen reader	Navigate, open film and delete confirmation	Visible focus and understandable labels
D08	Non-Nigerian test locale	Local date/number, captions, rights messaging	Accurate language/currency; unsupported region not marketed
D09	Server temporarily unavailable in staging	Refresh, sign in, play film	Friendly error, no broken access controls
D10	Valid purchase but user closes browser before redirect	Log in again after webhook	Correct single entitlement if provider settled

Performance and availability acceptance (ENGINEER)

Establish measured targets before choosing arbitrary numbers: first usable paint and player startup on low-end phones; 95th percentile API latency; crash-free sessions; error rate and uptime; peak viewers and movie byte-range requests; CDN egress cost; transcoding time per video hour; failed upload recovery; webhook retry queue; backup recovery time and point objective. Capture baseline from staging, tune caching without making masters public, and re-test. No load/fuzz test without an approved safe window and resource caps.

Phase 12 — Full project acceptance register

Each NOT RUN row is a real task for the employer or their engineer. **Do not mark a whole section complete because its source code compiles.**

ID	Section	Tester	Test / integration evidence needed	Status
A01	A	OWNER	Email verify/sign-in/logout with real test provider	NOT RUN
A02	A	ENGINEER	No seeded identity or admin takeover, verified binding review	NOT RUN
B01	B	ENGINEER	Two R2 buckets; private film direct URL inaccessible	NOT RUN
B02	B	ENGINEER	Independent FFmpeg preview + malware scan success/failure	NOT RUN
B03	B	OWNER	Anonymous preview vs purchased full film vs unpaid user	NOT RUN
B04	B	ENGINEER	Legacy public source migration and revocation	NOT RUN
C01	C	DBA	Upload-session SQL, RLS, quotas and two instance checks	NOT RUN
C02	C	OWNER	Creator approved upload queue admin publish	NOT RUN
C03	C	ENGINEER	500 MB multipart, corruption, interruption and cleanup	NOT RUN
D01	D	FINANCE	Legacy references/balances/payout independently reconciled	BLOCKED
D02	D	DBA	Isolated canonical SQL import and signed approval	BLOCKED
D03	D	OWNER	Test Paystack buy, cancellation and single entitlement	BLOCKED
D04	D	ENGINEER	Webhook authenticity, retry, race and atomic ledger	BLOCKED

ID	Section	Tester	Test / integration evidence needed	Status
D05	D	FINANCE	Completed refund, chargeback, bank payout & reconciliation flow	NOT BUILT
E01	E	OWNER	Web / purchases separate from reader /reader	NOT RUN
E02	E	OWNER	Reader no payment/external purchase promotion	NOT RUN
E03	E	OWNER	Account Settings delete with zero/positive balance	NOT RUN
E04	E	QA	Keyboard, VoiceOver/TalkBack, reduced motion, zoom	NOT RUN
F01	F	MOBILE	Native source built/signed on Android and iOS; native player/bookmark function on devices	NOT RUN
F02	F	MOBILE	System-browser Google OAuth + safe handoff (NOT BUILT); deep-link/navigation security on real phones	BLOCKED
F03	F	STORE OWNER	Play internal test/TestFlight and actual store policy review	BLOCKED
G01	G	OWNER	Report, block, moderator queue, copyright takedown	NOT BUILT
G02	G	LEGAL	Published support, policies and country rights	NOT BUILT
H01	H	ENGINEER	Staging and production deployments with separated secrets	NOT RUN
H02	H	OPS	Monitoring, signed CI/CD, restored backup, incident drill	NOT BUILT
I01	I	LEGAL	Territory + currency + tax + rights launch matrix	NOT BUILT
I02	I	QA	Localization/captions/accessible player per territory	NOT BUILT
J01	J	INDEPENDENT QA	Full regression and security testing with evidence	NOT RUN
J02	J	STORE OWNER	Store submission/review decision and signed release	NOT RUN

Blank test case form (copy for every row)

Test ID: _____ Date/time (WAT): _____ Tester: _____ Section: _____ Website/API or environment name: _____ Browser/phone and version: _____ Starting account (role only): _____ Steps I clicked: _____ Expected: _____ Actual: _____ Result: NOT RUN / PASS / FAIL / BLOCKED **Evidence filename/reference, without secrets: _____ Bug ticket/owner: _____ Retest date/result: _____ Reviewer sign-off: _____**

Phase 13 — Bugs, rollback, security incidents and handover

When something fails:

1. **STOP** repeating payments, destructive actions, uploads or administrative approvals. Write the URL, account **role**, time, screen, test ID, status and what happened. Do not share secrets or customer PII.
2. A red toast or error is not proof of data loss. Ask engineer to check **staging** app logs, Supabase logs, R2 object states and Paystack test references separately; use IDs, not raw card/identity information.
3. Create a bug ticket with severity: **Blocker** (money/identity/private film leak/account deletion), **High** (unusable core flow), **Medium** (major UX error), **Low** (cosmetic). Assign a person and expected retest date.
4. If an upload or payment partly succeeded, **do not retry blind**. First verify final database/provider state and request idempotent recovery. A second browser click must not produce duplicate money or rights.
5. To roll back, engineer disables new purchases/uploads at the gateway while preserving transaction receipts and canonical database records; revert app code only after deciding how pending jobs/wallet records continue to be reconciled. **Never restore stale wallet balances or revive the unsafe old JSON money-writing path.**

6. If an actual customer-data or financial incident occurs, activate the agreed company incident plan, secure evidence and notify qualified security/legal/finance contacts. The employer must not improvise a public announcement or delete logs.

Phase 14 — Final release sign-off sheet

All boxes must be documented, independently verified and approved; the current status of every box is NO.

- [ ] All 52 local automated checks continue to pass on the exact source (46 web/backend + 6 native policy checks); **separately** complete mobile SDK typecheck, signed builds and device tests.
- [ ] All real Supabase/R2/Paystack test integrations are connected, segregated and manually verified.
- [ ] Financial audit issues fully reconciled with signed documents; no demo money or unsupported payout migrated.
- [ ] Refunds, chargebacks, settlements and actual creator bank payouts have proven operational workflows and receipts.
- [ ] Paid films private and migrated, preview worker/scanner/queue stable and public legacy URLs revoked.
- [ ] Moderation/report/block, copyright rights, terms, privacy, support and deletion are fully built and reviewed.
- [ ] Native app provides independent useful functionality, system-browser OAuth, no prohibited digital purchase call-to-action.
- [ ] Global launch territories approved individually; legal/financial and accessibility issues resolved.
- [ ] Independent penetration test, load test, backup restore, logging and disaster recovery passed.
- [ ] Final manual case matrix all PASS or waived by an authorized owner with documented rationale (no security/financial blocker waived).
- [ ] Apple and Google have separately approved the submitted builds and metadata, if distributed through their stores.
- [ ] Employer, engineering lead, finance controller, privacy/legal owner and security reviewer have signed a production-release record.

Final release decision today: DO NOT LAUNCH. Qualify the newly supplied **F native source** on real devices, complete its secure system-browser OAuth, finish **G, H, I and J**, and close known **A–E operational/financial gates** first. This is a full guided **plan**, not a substitute for human testing or expert supervision of high-risk integrations.

Official help links (start here; never guess from random videos)

- Apple App Review Guidelines
- Apple reader-app entitlement
- Apple account deletion
- Google Play payments policy
- Google Play account deletion
- Supabase Auth SMTP
- Supabase Auth redirect URLs
- Cloudflare R2 setup
- Cloudflare R2 CORS
- Paystack Test Payments
- Paystack Webhooks
- Render Web Services
- Vercel Environment Variables